



Reinforcement Learning

B12901012 張維傑

B12901080 李俊翰

B12901185 謝承恩

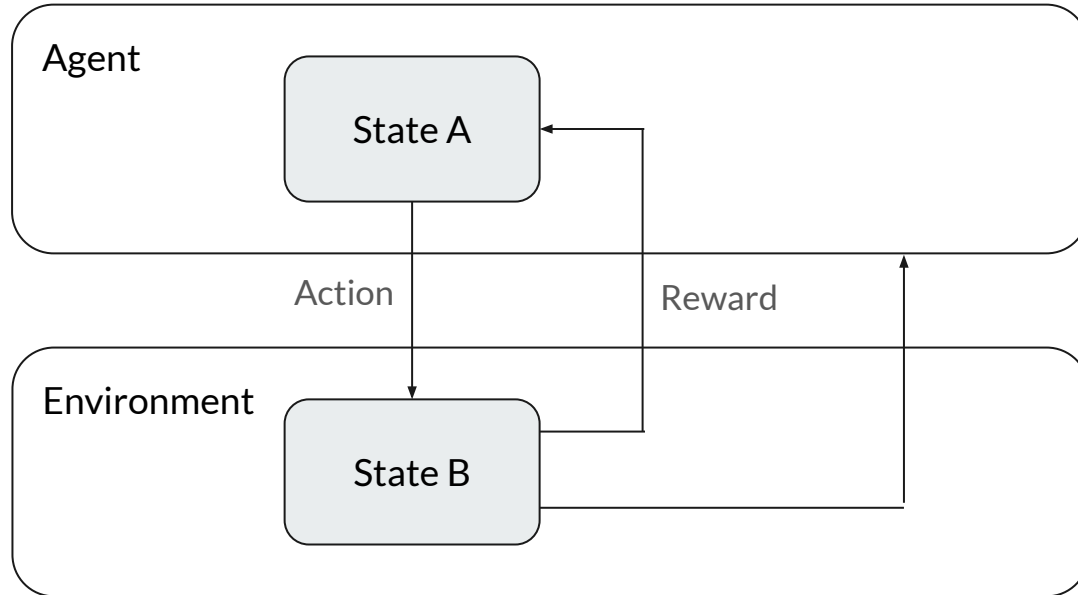


Outline

- Value-Based Learning
- Reward Shaping
- Policy-Based Learning
- MCTS
- AlphaZero
- Actor-Critic RL
- Learning GNN encoder + actor critic structure



Reinforcement Learning



Value-Based Learning

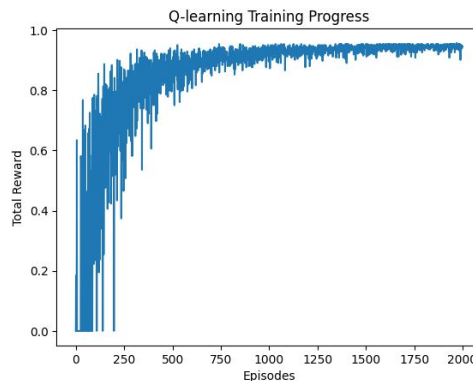
- Also known as **Q-Learning**
- **Bellman-Equation**: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t))$
- **Epsilon-Greedy**: For balancing the **exploration** and **exploitation**, set an epsilon value -> epsilon % randomly pick action, (1-epsilon) % pick the **highest value action**.

Algorithm Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Require: MDP with states $\mathcal{S}$ and actions $\mathcal{A}$

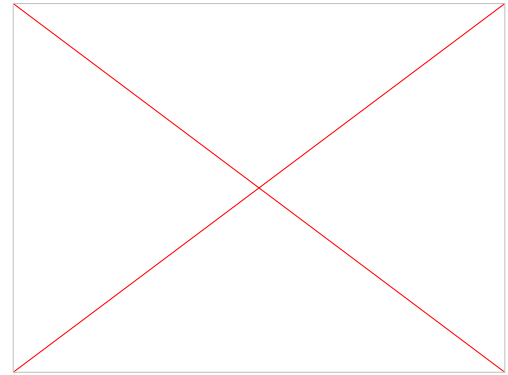
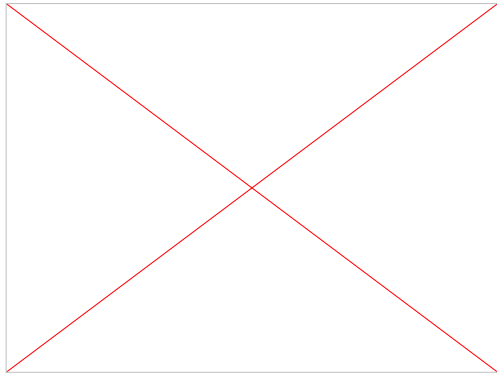
Ensure: Action-value function Q

```
1: Initialize  $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ , arbitrarily, and  $Q(\text{terminal-state}, \cdot) = 0$ 
   {Zero terminal values}
2: repeat
3:   Initialize  $s$  {Start new episode}
4:   repeat
5:     Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy) {Behavior
      policy}
6:     Take action  $a$ , observe  $r, s'$  {Environment step}
7:      $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_a Q(s', a) - Q(s, a)]$  {Q-learning update}
8:      $s \leftarrow s'$  {Move to next state}
9:   until  $s$  is terminal
10: until convergence
```



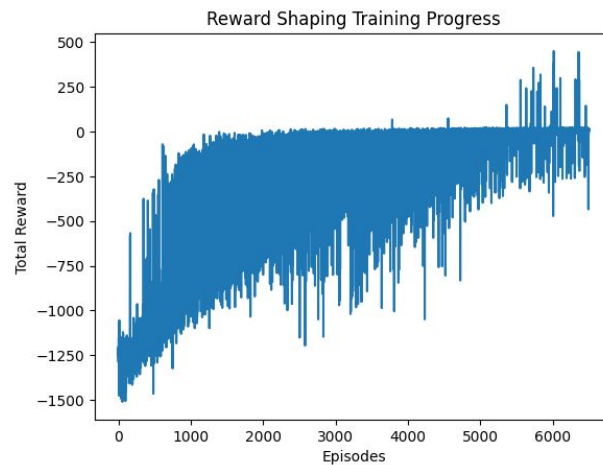


Value-Based Learning



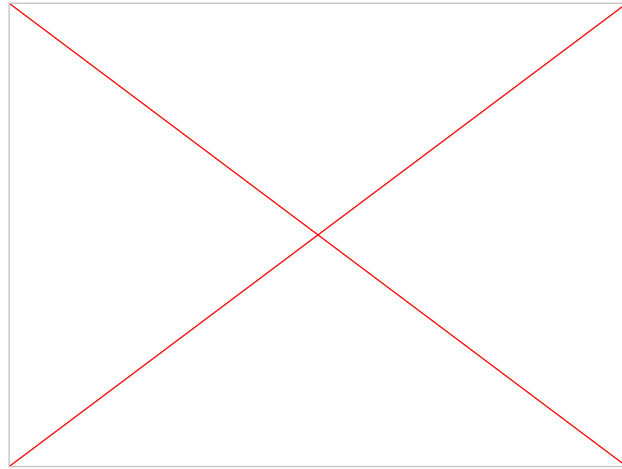
Reward Shaping

- Give a **small reward** when the agent achieve a small goal.
- Also give some **penalty** when the agent does some bad action.





Reward Shaping





Policy-Based Learning

- Action chosen by a **probability distribution**.
- Trace backward to modify the action probability distribution.

```
for t in reversed(range(len(trajectory))):
    state, action, reward = trajectory[t]

    # TODO: Compute discounted return.

    G = reward + gamma * G

    # TODO: Compute the policy gradient update for softmax logits

    pi = softmax(policy_table[state])
    grad = -pi.copy()
    grad[action] += 1.0
    policy_table[state] += alpha * G * grad
```



Monte Carlo Tree Search (MCTS)

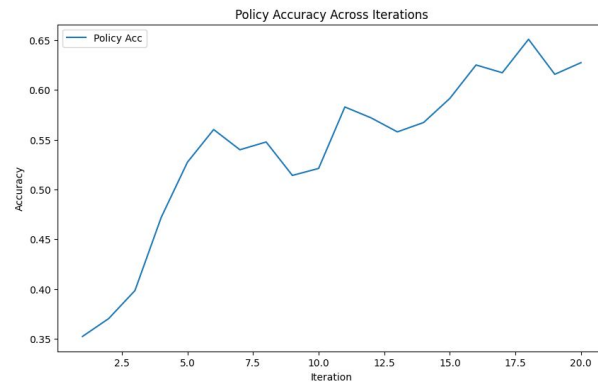
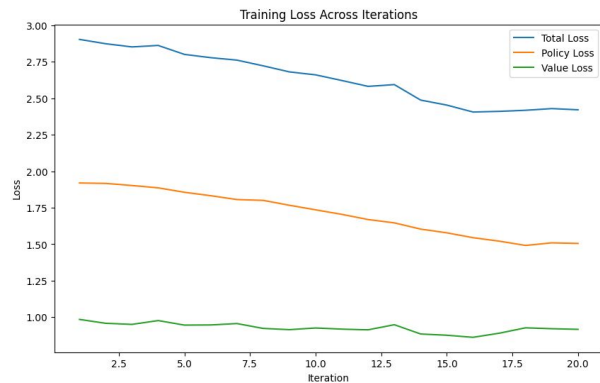
$$UCT(s, a) = Q(s, a) + C \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

- Predict the best move by exploring the future.
- **Selection:** select the most promising node to the leaf node by UCT value
- **Expansion:** expand the leaf node by adding unexpanded actions
- **Simulation:** perform a random rollout to estimate the value
- **Backpropagation:** update the values of the nodes on the path

Chosen action: 0
Root child statistics:
Action 0: visits = 19, value = -0.211
Action 1: visits = 2, value = 1.000
Action 2: visits = 2, value = 1.000
Action 3: visits = 4, value = 0.500
Action 4: visits = 14, value = -0.286
Action 5: visits = 3, value = 0.667
Action 6: visits = 6, value = 0.333

AlphaZero

- **Dual-Headed CNN Architecture:** Single network outputs `pred_policy` and `pred_value`
- **Apply MCTS:** Generating `target_policy` and `target_value`
- **Comparison to improve**





Value-Based and Policy-Based RL

- Downsides of Value-Based RL
 - Unable to handle continuous action space
- Downsides of Policy-Based RL
 - One bad move penalize the whole sequence of action
 - High variance: Cannot differentiate a lucky win from a good strategy



Actor-Critic RL

- Split into two NNs: The Actor and The Critic
- The actor
 - Policy-Based
 - Decides the action
- The critic
 - Value-Based
 - Judge the score of a state



Actor-Critic RL: Advantage Function

- The critic evaluates the **Advantage Function**
- $\text{advantage} = \text{actual_reward} - \text{critic_expectation}$
- Advantage used as the learning signal for the actor



Actor-Critic RL: Bootstrapping

- The sparse reward problem: In games like chess, reward is only given at the final time step
- Calculate the advantage function by critic's score
- Example:
 - Chess winrate dropped from 80% to 20% after a move
 - $\text{Advantage} = (20\% - 80\%) * \text{some_constant}$
 - Train the actor to avoid this behavior



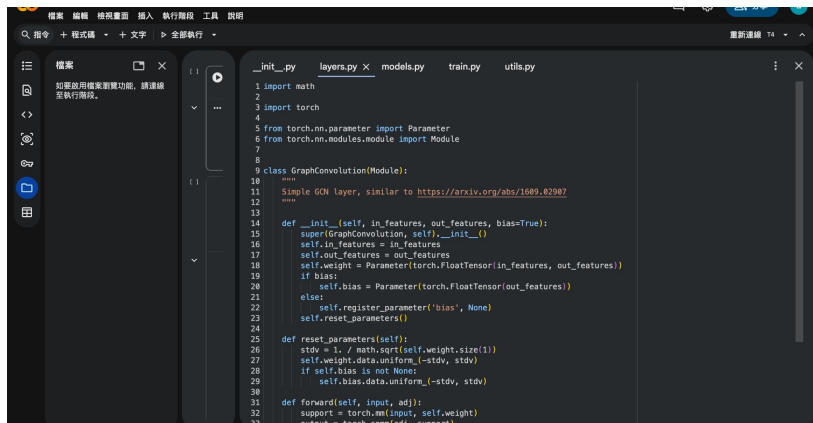
Actor-Critic RL: Updating Critic

- Supervised learning
- Input: The current state of the environment
- Output target: The true value of the state (immediate reward + future expectation)

Trying GNN

<https://github.com/tkipf/pygcn>

Thomas Kipf, [Graph Convolutional Networks](#)
(2016)



```
__init__.py layers.py models.py train.py utils.py
1 import math
2
3 import torch
4
5 from torch.nn.parameter import Parameter
6 from torch.nn.modules.module import Module
7
8 class GraphConvolution(Module):
9     """
10     Simple GCN layer, similar to https://arxiv.org/abs/1609.02907
11     """
12     def __init__(self, in_features, out_features, bias=True):
13         super(GraphConvolution, self).__init__()
14         self.in_features = in_features
15         self.out_features = out_features
16         self.weight = Parameter(torch.FloatTensor(in_features, out_features))
17         if bias:
18             self.bias = Parameter(torch.FloatTensor(out_features))
19         else:
20             self.register_parameter('bias', None)
21         self.reset_parameters()
22
23     def reset_parameters(self):
24         stdv = 1. / math.sqrt(self.weight.size(1))
25         self.weight.data.uniform_(-stdv, stdv)
26         if self.bias is not None:
27             self.bias.data.uniform_(-stdv, stdv)
28
29     def forward(self, input, adj):
30         support = torch.mm(input, self.weight)
31         output = torch.matmul(support, adj)
```

```
Epoch: 0176 loss_train: 0.4203 acc_train: 0.9357 loss_val: 0.7265 acc_val: 0.8133 time: 0.0262s
Epoch: 0177 loss_train: 0.4406 acc_train: 0.9429 loss_val: 0.7263 acc_val: 0.8167 time: 0.0270s
Epoch: 0178 loss_train: 0.4192 acc_train: 0.9429 loss_val: 0.7245 acc_val: 0.8167 time: 0.0278s
Epoch: 0179 loss_train: 0.4015 acc_train: 0.9571 loss_val: 0.7227 acc_val: 0.8133 time: 0.0262s
Epoch: 0180 loss_train: 0.4099 acc_train: 0.9429 loss_val: 0.7205 acc_val: 0.8133 time: 0.0266s
Epoch: 0181 loss_train: 0.4052 acc_train: 0.9429 loss_val: 0.7186 acc_val: 0.8167 time: 0.0303s
Epoch: 0182 loss_train: 0.4618 acc_train: 0.9500 loss_val: 0.7168 acc_val: 0.8100 time: 0.0267s
Epoch: 0183 loss_train: 0.4262 acc_train: 0.9429 loss_val: 0.7152 acc_val: 0.8167 time: 0.0262s
Epoch: 0184 loss_train: 0.3892 acc_train: 0.9571 loss_val: 0.7136 acc_val: 0.8167 time: 0.0263s
Epoch: 0185 loss_train: 0.3913 acc_train: 0.9643 loss_val: 0.7130 acc_val: 0.8133 time: 0.0266s
Epoch: 0186 loss_train: 0.3952 acc_train: 0.9214 loss_val: 0.7128 acc_val: 0.8133 time: 0.0280s
Epoch: 0187 loss_train: 0.4407 acc_train: 0.9286 loss_val: 0.7131 acc_val: 0.8133 time: 0.0264s
Epoch: 0188 loss_train: 0.4304 acc_train: 0.9500 loss_val: 0.7122 acc_val: 0.8100 time: 0.0284s
Epoch: 0189 loss_train: 0.4197 acc_train: 0.9500 loss_val: 0.7109 acc_val: 0.8167 time: 0.0302s
Epoch: 0190 loss_train: 0.4230 acc_train: 0.9214 loss_val: 0.7112 acc_val: 0.8133 time: 0.0294s
Epoch: 0191 loss_train: 0.4447 acc_train: 0.9071 loss_val: 0.7129 acc_val: 0.8067 time: 0.0262s
Epoch: 0192 loss_train: 0.4565 acc_train: 0.9214 loss_val: 0.7144 acc_val: 0.8100 time: 0.0317s
Epoch: 0193 loss_train: 0.4044 acc_train: 0.9643 loss_val: 0.7147 acc_val: 0.8067 time: 0.0302s
Epoch: 0194 loss_train: 0.4392 acc_train: 0.9357 loss_val: 0.7137 acc_val: 0.8067 time: 0.0269s
Epoch: 0195 loss_train: 0.3888 acc_train: 0.9429 loss_val: 0.7095 acc_val: 0.8067 time: 0.0282s
Epoch: 0196 loss_train: 0.4130 acc_train: 0.9286 loss_val: 0.7058 acc_val: 0.8100 time: 0.0264s
Epoch: 0197 loss_train: 0.3710 acc_train: 0.9571 loss_val: 0.7025 acc_val: 0.8200 time: 0.0260s
Epoch: 0198 loss_train: 0.4163 acc_train: 0.9500 loss_val: 0.6996 acc_val: 0.8167 time: 0.0268s
Epoch: 0199 loss_train: 0.3710 acc_train: 0.9571 loss_val: 0.6983 acc_val: 0.8200 time: 0.0279s
Epoch: 0200 loss_train: 0.3815 acc_train: 0.9429 loss_val: 0.6974 acc_val: 0.8167 time: 0.0321s
Optimization Finished!
Total time elapsed: 4.7435s
Test set results: loss= 0.7204 accuracy= 0.8340
```

Trying GNN

```
1 import numpy as np
2
3 # 1. 建立 4x4 的全零矩陣
4 A = np.zeros((4, 4))
5
6 # 2. 填入連線關係 (無向圖, 所以要雙向設定)
7 # A 連 B (0 連 1)
8 A[0, 1] = 1
9 A[1, 0] = 1
10
11 # B 連 C (1 連 2)
12 A[1, 2] = 1
13 A[2, 1] = 1
14
15 # C 連 D (2 連 3)
16 A[2, 3] = 1
17 A[3, 2] = 1
18
19 print("=== 1. 原始相鄰矩陣 A ===")
20 print(A)
```

```
23 # 加上 4x4 的單位矩陣 (對角線為 1)
24 I = np.eye(4)
25 A_tilde = A + I
26
27 print("\n=== 2. 加上自迴圈的相鄰矩陣 A_tilde ===")
28 print(A_tilde)
29
30 # 沿著列 (axis=1) 加總, 算出每個節點的 Degree
31 degree_array = np.sum(A_tilde, axis=1)
32 print("\n=== 3a. 每個節點的 Degree ===")
33 print(degree_array)
34 # 預期結果: A有2個(自己+B), B有3個(自己+A+C)...
35
36 # 取倒數 ( $D^{-1}$ )
37 degree_inv_array = 1.0 / degree_array
38 print("\n=== 3b. Degree 的倒數 ===")
39 print(degree_inv_array)
40
41 # 把這個一維陣列變成對角矩陣 (這對應到 sp.diags)
42 D_inv = np.diag(degree_inv_array)
43 print("\n=== 3c. 倒數 Degree 矩陣 D_inv ===")
44 print(D_inv)
45
46 # 矩陣相乘: D_inv 乘上 A_tilde
47 normalized_A = np.dot(D_inv, A_tilde)
48
49 print("\n=== 4. 最終正規化後的相鄰矩陣 ===")
50 print(normalized_A)
```

Trying GNN

```
53 # === 第五步：建立特徵矩陣 (Features) ===
54 # 假設我們每個節點有 2 個特徵。
55 X = np.array([
56     [10, 20],      # 節點 A 的特徵
57     [100, 200],    # 節點 B 的特徵
58     [1000, 2000],  # 節點 C 的特徵
59     [10000, 20000] # 節點 D 的特徵
60 ])
61
62 print("\n=== 5. 原始特徵矩陣 X ===")
63 print(X)
64
65 # === 第六步 — 訊息傳遞 (Message Passing) ===
66 # 把「正規化後的相鄰矩陣」乘上「特徵矩陣」
67 # 數學上這就是  $D^{-1} * A\_tilde * X$ 
68 new_X = np.dot(normalized_A, X)
69
70 print("\n=== 6. 聚合鄰居資訊後的新特徵 new_X ===")
71 print(new_X)
```

```
=== 1. 原始相鄰矩陣 A ===
[[0. 1. 0. 0.]
 [1. 0. 1. 0.]
 [0. 1. 0. 1.]
 [0. 0. 1. 0.]]

=== 2. 加上自迴圈的相鄰矩陣 A_tilde ===
[[1. 1. 0. 0.]
 [1. 1. 1. 0.]
 [0. 1. 1. 1.]
 [0. 0. 1. 1.]]

=== 3a. 每個節點的 Degree ===
[2. 3. 3. 2.]

=== 3b. Degree 的倒數 ===
[0.5      0.33333333 0.33333333 0.5      ]

=== 3c. 倒數 Degree 矩陣 D_inv ===
[[0.5      0.      0.      0.      ]
 [0.      0.33333333 0.      0.      ]
 [0.      0.      0.33333333 0.      ]
 [0.      0.      0.      0.5      ]]

=== 4. 最終正規化後的相鄰矩陣 ===
[[0.5      0.5      0.      0.      ]
 [0.33333333 0.33333333 0.33333333 0.      ]
 [0.      0.33333333 0.33333333 0.33333333]
 [0.      0.      0.5      0.5      ]]

=== 5. 原始特徵矩陣 X ===
[[ 10   20]
 [ 100  200]
 [1000 2000]
 [10000 20000]]

=== 6. 聚合鄰居資訊後的新特徵 new_X ===
[[ 55.   110.]
 [ 370.  740.]
 [ 3700. 7400.]
 [ 5500. 11000.]]
```



Learning the Strategy: GNN Encoder + RL

- Reviewing the reference(google): using grid?
- What I want: 2D coordinates
- the method: GNN Encoder + RL



Learning the Strategy: GNN Encoder + RL

1. 神經網路定義 (GNN Encoder & Actor & Critic)
2. 強化學習代理人 (SAC Agent 學習升級版) (`__init__`, `select_action`, `update`)
- 2.5 經驗回放池 (Replay Buffer)
3. 確保環境加上畫布邊界與Reward 限制
4. 解析器(讀取題目給定的training data) (部分讀取)
5. 視覺化工具
6. 主訓練迴圈



Learning the Strategy: GNN Encoder + RL

6. 主訓練迴圈

- 觀察與決策 (select_action): AI 觀察當前的畫布狀態 (哪些放了？放在哪？), 經過GNN 思考後, 輸出一個座標比例(x,y)。
- 物理執行 (env.step): 環境接收座標, 將其還原為真實物理座標。
- 獎勵回饋 (Reward Calculation): 環境立刻檢查: 這顆模塊有沒有壓到別人？離它的鄰居是不是太遠？然後給出一個分數。
- 存入記憶 (push): 將這一步的「前因後果」存入Replay Buffer。這非常關鍵, 因為AI 需要回頭複習這些經驗。

Learning the Strategy: GNN Encoder + RL

```
# =====  
# 1. 神經網路定義 (GNN Encoder & Actor)  
# =====  
class DummyGNNEncoder(nn.Module):  
    def __init__(self, in_dim, out_dim):  
        super().__init__()  
        self.fc = nn.Linear(in_dim, out_dim)  
  
    def forward(self, node_feat, adj):  
        x = self.fc(node_feat)  
        g = x.mean(dim=1)  
        return x, g
```

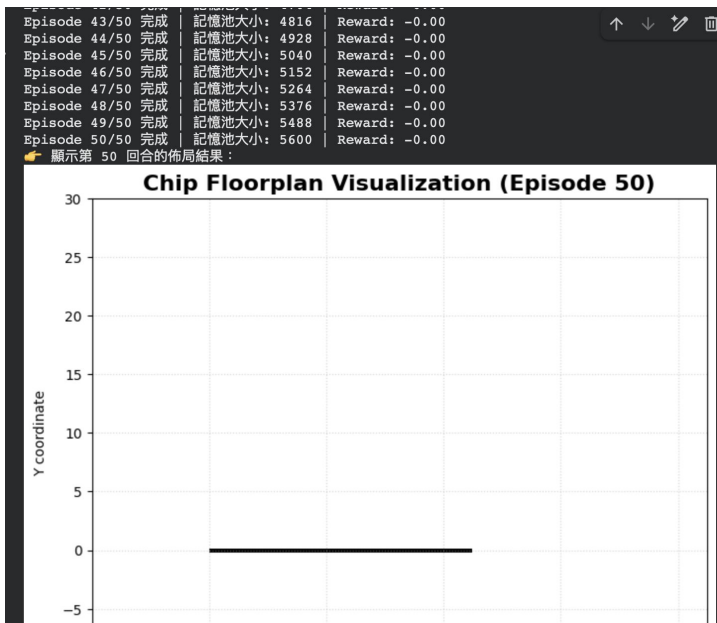
```
29 class DummyActor(nn.Module):  
30     def __init__(self, in_dim):  
31         super().__init__()  
32         self.net = nn.Sequential([  
33             nn.Linear(in_dim, 32),  
34             nn.ReLU(),  
35             nn.Linear(32, 2),  
36             nn.Sigmoid() #Sigmoid  
37         ])  
38  
39     def forward(self, g):  
40         action = self.net(g)  
41         return action, None
```



Learning the Strategy: GNN Encoder + RL

```
43 class DummyCritic(nn.Module):
44     def __init__(self, in_dim):
45         super().__init__()
46         # 接收 GNN 的特徵 (16維) + 動作座標 x,y (2維) = 18維
47         self.net = nn.Sequential(
48             nn.Linear(in_dim + 2, 32),
49             nn.ReLU(),
50             nn.Linear(32, 1) # 最終輸出一個「分數」(Q-value)
51         )
52
53     def forward(self, g, action_xy):
54         # 將特徵與動作合併起來一起評分
55         x = torch.cat([g, action_xy], dim=-1)
56         q_value = self.net(x)
57         return q_value
58
```

Learning the Strategy: GNN Encoder + RL





Learning the Strategy: GNN Encoder + RL

GNN 的軟肋:

Overlap-Free Constraint (Area 必須為 0)

痛點: GNN 只懂拓撲, 不懂幾何排斥。GNN 算出兩個 Block 關係極好, 可能會把它們的(x,y) 座標預測在同一個點上, 直接導致Overlap 違規。

Soft-block 的形狀變形 (Area 容忍度 $\leq 1\%$)

痛點: 這裡的 Block 不是固定大小的死磚頭, 而是可以改變長寬比的 Soft-block (只要面積誤差小於 1%)。GNN 的數學運算中, 很難原生地去處理這種連續的長寬連續變形 ($w_i \times h_i$) 與面積守恒律。

詭異的 Soft Constraints (Grouping, MIB, Boundary)

痛點: 要把特定的 Block 貼在 Bounding-box 的邊界上, 或是要求幾個 Block 必須緊緊相連 (Abutted)。這些是純粹的「空間幾何限制」, GNN 的特徵向量無法直接「強制」實體佈局滿足這些條件。公式中 $\beta \cdot \text{Violations}$ 的指數懲罰, 會讓任何空間感不好的模型分數暴跌。



Reference

- DRL課程作業HW1、HW2
- Gemini
- <https://github.com/tkipf/pygcn>
- Achiam, J. (2018). Spinning Up in Deep Reinforcement Learning. OpenAI